

FACULTY DEVELOPMENT PROGRAM

LLM Fine-Tuning: Concepts, Methods & Training Mechanics

From pretrained language models to parameter-efficient adaptation

Session 1

Learning Roadmap

1

LLMs: what they are and how they work

Transformer architecture, tokens and autoregressive generation.

2

Why fine-tuning?

When and why we adapt a pretrained model at all.

3

Full fine-tuning vs prompt engineering vs prompt tuning

Comparing the adaptation spectrum and its trade-offs.

4

Parameter-efficient fine-tuning: LoRA & QLoRA

Low-rank adapters and quantized backbones.

5

Supervised fine-tuning (SFT)

Teaching a model from labeled input/output examples.

6

Training mechanics, loss and optimization

Forward pass, backpropagation and gradient descent.

7

Evaluation, overfitting and practical considerations

Judging whether fine-tuning actually worked.

Learning Outcomes

1

Basic architecture and training paradigm of LLMs.

Tokens, embeddings and Transformer blocks.

2

Distinguish prompt engineering, prompt tuning, full fine-tuning and PEFT.

Know which parameters each method touches.

3

Why LoRA and QLoRA reduce the cost of adapting LLMs.

Low-rank matrices and quantized weights.

4

Understand SFT, tokenization, loss, backpropagation and optimizer updates.

The full supervised training loop.

5

Interpret training/validation behavior and select appropriate evaluation strategies.

Spot overfitting before it costs you.

6

Connect fine-tuning decisions to practical deep-learning applications.

Translate theory into workshop practice.

Examples of Modern LLM Families

Different organizations and model families; capabilities and deployment options vary

MODEL FAMILIES YOU MAY ENCOUNTER

**OpenAI
GPT**

**Anthropic
Claude**

**Google
Gemini / Gemma**

**Meta
Llama**

**Alibaba
Qwen**

**Mistral
Mistral**

WHAT DIFFERS?

Model size

Training data

Architecture / variants

Context length

Modalities

Licensing / access

Deployment options

Hands-on: Qwen2.5-0.5B-Instruct — small enough for Colab, suitable for SFT + LoRA.

What Is an LLM?



A neural language model

Trained on large-scale text to predict and generate sequences of tokens, one step at a time.



Transformer-based

Modern LLMs are predominantly Transformer-based and use self-attention to model contextual relationships between tokens.



Broad pretraining, narrow adaptation

Pretraining creates broad language and representation capabilities; downstream adaptation specializes behavior for a task.



Autoregressive generation

LLMs generate text autoregressively — repeatedly predicting the next token conditioned on everything generated so far.

From Text to Tokens



Tokenization

Text is converted into tokens using a tokenizer; each token is mapped to a unique integer ID from the model's vocabulary.



Sub-word units

Tokens may represent whole words, sub-words or punctuation — not necessarily complete words.



Embeddings

Token IDs are transformed into dense vector embeddings before entering the Transformer stack.



Sequence length matters

Longer sequences directly increase memory and compute requirements during training and inference.



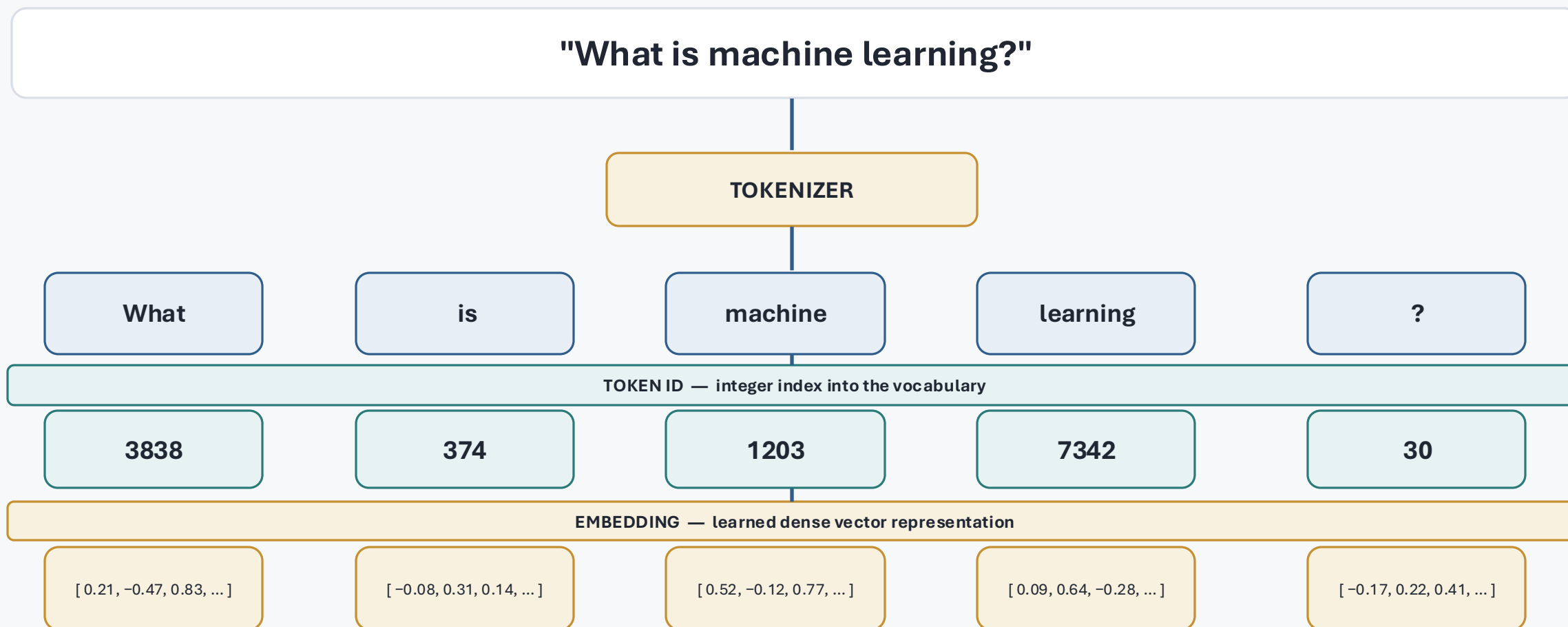
Special tokens

Special tokens and model-specific formatting (e.g. chat markers) matter for instruction-tuned models.

What Does an LLM Actually See?

Illustrative values

One sentence → tokens → token IDs → embedding vectors



KEY DISTINCTION • Token ID = index • Embedding = learned numerical vector

Transformer: High-Level View



- Each block combines self-attention and a feed-forward network with residual connections and normalization.
- Self-attention lets every token pull information from other relevant tokens in its context.
- The final vocabulary logits are converted into probabilities for next-token prediction.

Self-Attention: Core Idea

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$



Query, Key, Value

Each token produces Query (Q), Key (K) and Value (V) representations via learned projections.



Multi-head attention

Multiple attention heads learn different relational patterns in parallel, then are combined.



Attention scores

Scores estimate how strongly tokens should attend to one another based on Q-K similarity.



Causal masking

Masking prevents a decoder-only LLM from using future tokens while generating text.



Scaled dot-product

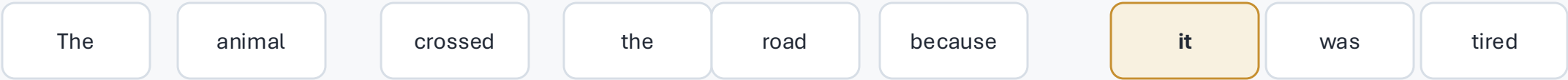
Scores are scaled by $\sqrt{d_k}$ for stability, then softmax-normalized into attention weights.

Self-Attention — Visual Walkthrough

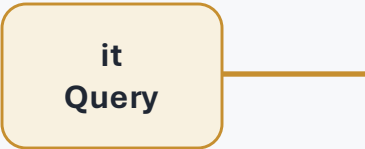
Storyboard for an animation/GIF: how one token gathers context

Animation storyboard

"The animal crossed the road because it was tired."



1. Focus on current token



2. Compare Query with Keys of all tokens

3. Attention / relevance scores



4. Weighted Values → output

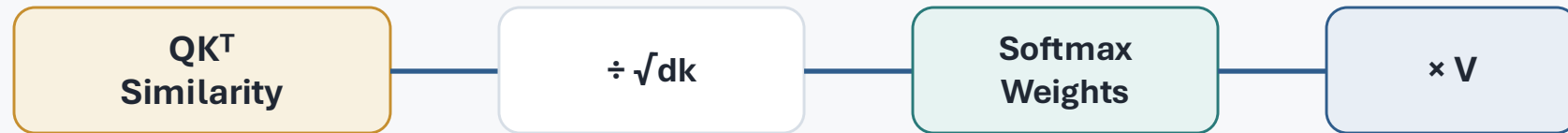
Attention weights × Value vectors → contextualized representation of “it”

Self-Attention — $Q, K, V \rightarrow \text{Attention} \rightarrow \text{Output}$

From learned projections to a contextualized token representation



Q and K determine relevance; V carries the information to combine



Contextualized output

Illustrative weights
animal 0.60
road 0.08
tired 0.32

Key idea:

The model computes a weighted combination of information from multiple tokens.

Pretraining vs Instruction Tuning



Pretraining

- Learns general language patterns from large, diverse corpora.
- Typically trained with next-token prediction as the sole objective.
- Produces broad, general-purpose representations — not task-specific behavior.



Instruction Tuning

- Adapts a pretrained model to follow human instructions.
- Trained on curated instruction/response pairs to shape useful behavior.
- Produces a model that responds helpfully rather than just completing text.

Fine-tuning is an adaptation stage — not training an LLM from scratch. Different adaptation methods change different parts of the learning process.

What Is Fine-Tuning?



Continued training

Fine-tuning continues training from a pretrained model on a smaller, task- or domain-specific dataset.



Adapts behavior

The objective is to adapt model behavior, task performance, style or domain response patterns.



Full or partial updates

Fine-tuning can update all parameters (full fine-tuning) or only a small trainable subset (PEFT).



Outcome depends on setup

Data quality, learning rate, number of epochs and evaluation design strongly affect the outcome.

Why Fine-Tuning?



Specialize a general model

Adapt a general model to a specialized task or a specific response format.



Improve instruction following

Improve instruction following and domain-specific behavior beyond what prompting alone achieves.



Consistent output style

Learn consistent terminology, structure or output style across many requests.



Shorter prompts at inference

Reduce dependence on very long, repeated prompts for patterns the model has now internalized.



Best with strong examples

Most useful when the desired behavior can be represented clearly by high-quality training examples.

Full Fine-Tuning vs Prompt Engineering vs Prompt Tuning



Full Fine-Tuning

- Updates most or all model weights using task-specific data.
- Maximum flexibility; highest compute and storage cost.



Prompt Engineering

- Keeps parameters fixed; improves instructions/context at inference.
- Zero training cost, but limited by context window.



Prompt Tuning

- Learns a small set of continuous prompt embeddings.
- Base model stays completely frozen.



LoRA / PEFT

- Learns small trainable updates; pretrained weights frozen.
- Our hands-on focus for Session 2.

Choice depends on task, data, compute budget, desired behavior and deployment constraints.

Prompt Engineering — Change the Input, Not the Model

Same model parameters; different instructions

WEAK PROMPT

Explain backpropagation.

Generic / unconstrained answer

STRUCTURED PROMPT

You are a Deep Learning instructor.
Explain backpropagation to a beginner.
Use: definition → intuition → math → example →
misconception → takeaway.

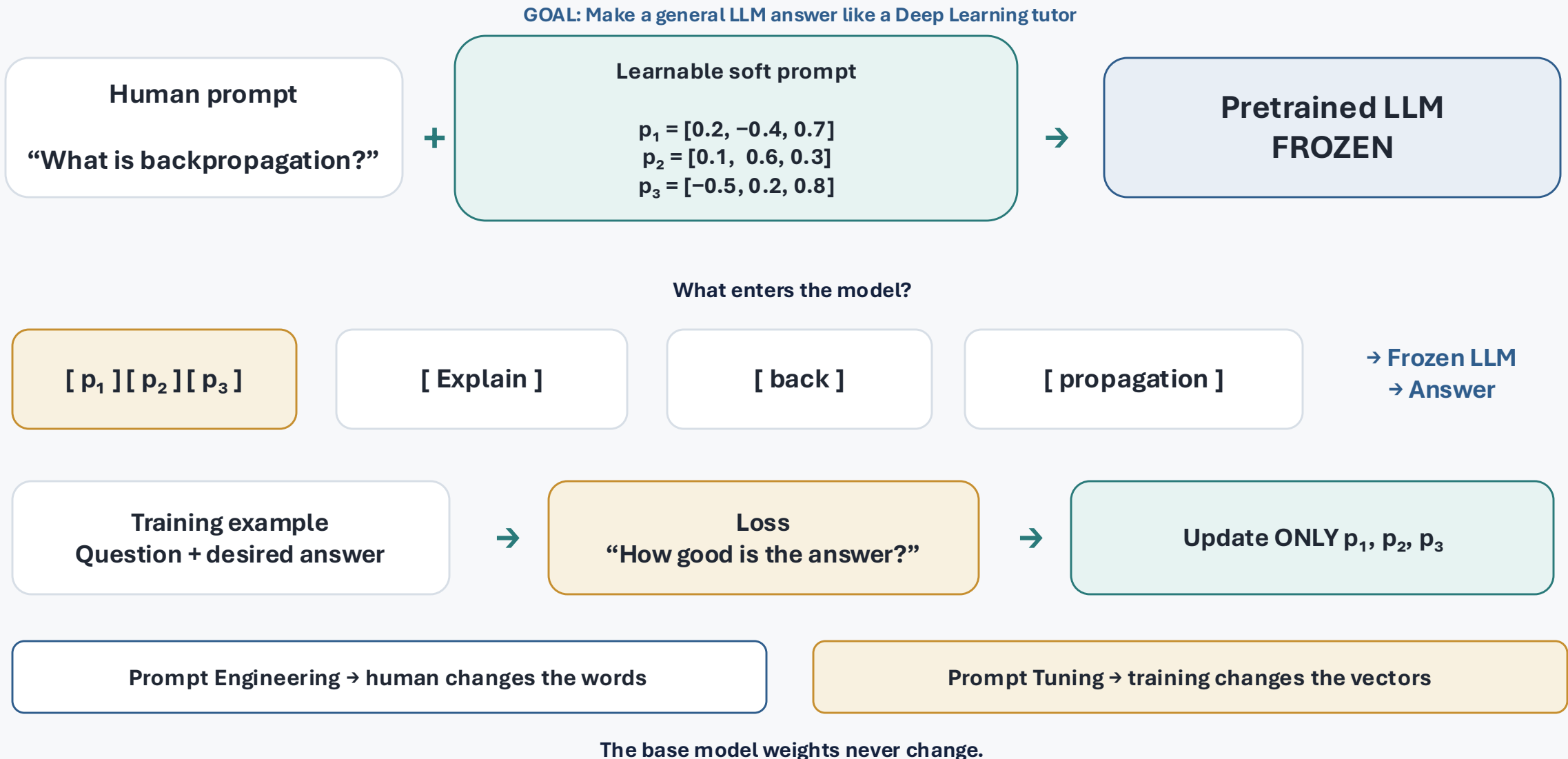
Structured response

- 1. Definition**
- 2. Intuition**
- 3. Mathematical idea**
- 4. Example**
- 5. Misconception**
- 6. Takeaway**

Prompt engineering changes the INPUT. Model weights remain unchanged.

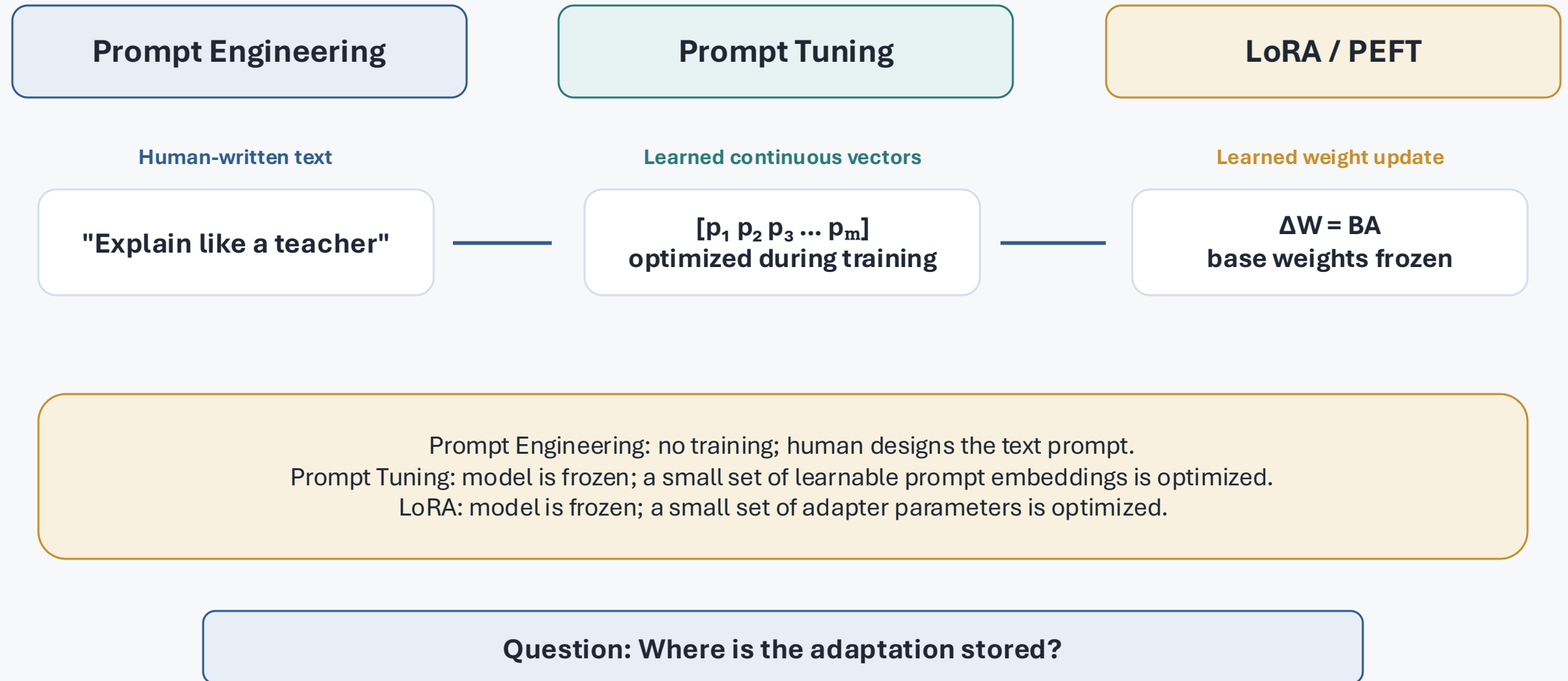
Prompt Tuning — A Simple Example

Instead of learning words, we learn a few small vectors that steer a frozen model.



Prompt Tuning — Learning the Prompt

Human-written prompts vs learned soft prompts



Full Fine-Tuning



Start from pretrained weights

Optimize every model parameter directly on the target dataset.



Expensive at scale

Maximum flexibility, but optimizer states, gradients and parameter storage can be very costly.



Substantial GPU demand

Can require large GPU memory and careful hyperparameter tuning to train stably.



Forgetting risk

Risk of catastrophic forgetting or overfitting increases with small or narrow datasets.



When it is justified

Useful when sufficient compute/data are available and deep adaptation is genuinely required.

Parameter-Efficient Fine-Tuning (PEFT)



Freeze most parameters

PEFT freezes most pretrained parameters and trains only a small number of additional or modified parameters.



Key benefits

Lower memory use, faster experimentation, and much smaller task-specific artifacts to store and share.



Common approaches

LoRA, adapters, and prompt-based methods are the most widely used PEFT techniques.



Still needs rigor

PEFT does not remove the need for good data and rigorous evaluation — it only reduces compute cost.

LoRA — Low-Rank Adaptation

$$W' = W + BA \quad (\Delta W \approx B \cdot A, W \text{ frozen})$$



Low-rank approximation

Instead of learning a full weight update ΔW , LoRA approximates it as the product of two small matrices, $B \cdot A$.



Frozen base weights

W is frozen; only the small A and B matrices are trainable, drastically cutting parameter count.



Rank controls capacity

Rank r controls the dimensionality — and therefore the capacity — of the low-rank update.



Where it's inserted

Commonly inserted into attention projection layers such as q_proj , k_proj , v_proj and o_proj .



Portable adapters

Adapters can be stored separately from the base model — small files, easy to share and swap.

LoRA: What Does $B \times A$ Actually Mean?

The low-rank update uses fewer trainable parameters than the full weight update.

$$W' = W + \Delta W \quad \text{where} \quad \Delta W = B A$$

FULL FINE-TUNING

$$\Delta W$$
$$1000 \times 1000$$

1,000,000 trainable values

LoRA — RANK $r = 8$

$$B$$
$$1000 \times 8$$

$$A$$
$$8 \times 1000$$

$$BA$$
$$1000 \times 1000$$

8,000 + 8,000 = 16,000 trainable values

Small numerical example — choose $r = 1$ so the parameter reduction is visible

$$B =$$
$$\begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix}$$

$$A =$$
$$\begin{bmatrix} 1 & 4 & 2 \end{bmatrix}$$

$\times$

$$BA =$$
$$\begin{bmatrix} 2 & 8 & 4 \\ 3 & 12 & 6 \\ 1 & 4 & 2 \end{bmatrix}$$

B: 3×1
A: 1×3
= 6 params

BA: 3×3
= 9 values

KEY IDEA: 3×3 full update = 9 params; rank-1 LoRA uses $3 + 3 = 6$ trainable params.

LoRA: What the Rank Means



- Rank r controls the dimensionality of the low-rank update — the core LoRA hyperparameter.
- Lower rank $\rightarrow$ fewer trainable parameters and lower adaptation capacity, but faster and lighter.
- Higher rank $\rightarrow$ more capacity, but greater memory/storage cost and potentially higher overfitting risk.
- LoRA alpha scales the contribution of the adapter; LoRA dropout adds further regularization.

QLoRA



Quantized base + LoRA

QLoRA combines quantized base-model weights (often 4-bit) with trainable LoRA adapters on top.



Lower-precision backbone

The base model is kept in a lower-precision representation while only the LoRA parameters are trained in higher precision.



Reduced memory footprint

This reduces memory requirements substantially, making larger models accessible on constrained GPUs.



Three moving parts

Typical components: low-bit quantization, LoRA adapters, and standard supervised fine-tuning on top.



When to reach for it

Especially useful when full-precision loading of the base model is too memory-intensive for your hardware.

Supervised Fine-Tuning (SFT)



Known input/output pairs

SFT trains the model on input/target examples where the desired assistant response is already known.



Role-structured examples

For conversational datasets, examples commonly contain system, user and assistant roles.



High-probability targets

The model learns to assign high probability to the target response tokens given the conversation context.



Loss drives updates

Loss is computed from token-level predictions and backpropagated through the trainable parameters.



Compatible with LoRA

With LoRA, the base model stays frozen while only the adapter parameters receive updates.

SFT Data Flow



- Backpropagation calculates gradients for every trainable parameter from that loss.
- The optimizer then updates the trainable weights; validation data monitors generalization along the way.

Training Mechanics: Forward Pass → Loss



- Cross-entropy / negative log-likelihood measures disagreement with the target tokens.
- Lower loss means the model assigns higher probability to the observed target sequence.
- Loss is a training signal — not a complete measure of real-world usefulness.

Backpropagation & Gradient Descent



- Backpropagation applies the chain rule to compute gradients of the loss with respect to every trainable parameter.
- Gradients indicate how a small parameter change would affect the loss; the optimizer uses them to update parameters.
- With LoRA, gradients update only the adapter parameters — the base weights remain frozen throughout.
- Learning rate controls the scale of each optimizer update — too high overshoots, too low stalls training.

Key Training Hyperparameters

- 

Learning rate
Controls the magnitude of each parameter update — the single most sensitive hyperparameter.
- 

Epochs
Number of complete passes through the training data.
- 

Batch size
Number of examples processed per forward/backward pass before an update.
- 

LoRA rank, alpha, dropout
Control adapter capacity and regularization strength.
- 

Gradient accumulation
Simulates a larger effective batch using several smaller, memory-friendly batches.
- 

Sequence length
Affects compute and memory consumption for every batch.

Training vs Validation



Healthy Pattern

- Training loss decreases as parameters update.
- Training data contributes directly to updates.
- Train ↓ + validation ↓ is generally encouraging.



Overfitting Pattern

- Validation data never contributes to updates — it only measures generalization.
- Validation loss stops improving or starts rising.
- Train ↓ + validation ↑ can indicate overfitting.

Validation performance should always be interpreted together with task-specific output quality — not loss numbers alone.

Evaluating a Fine-Tuned LLM



Intrinsic metrics

Training loss, validation loss and perplexity — fast, automatic, but indirect signals.



Task metrics

Accuracy, F1, exact match or other task-specific measures where ground truth is well-defined.



Qualitative evaluation

Correctness, relevance, clarity, consistency and style — read the outputs, not just the numbers.



Human evaluation

Especially valuable for open-ended generation where there is no single correct answer.



LLM-as-a-judge

Can scale evaluation efficiently, but introduces its own biases and should be validated against humans.

Fine-Tuning vs RAG vs Prompting



Prompt Engineering

- Change instructions/context without any training.
- Fastest to iterate; limited by context window.



RAG

- Retrieve external knowledge, feed it to the model at inference time.
- Best for fast-changing or proprietary knowledge.



Fine-Tuning

- Adapt behavior, task execution or response style through training.
- Best for consistent style, format or specialized skill.

RAG and fine-tuning can be combined for systems needing both reliable external knowledge and specialized behavior. Choose the simplest approach that reliably solves the problem.

Session 1 — Key Takeaways

- ✓ LLMs are pretrained general-purpose models that can be adapted for specialized behavior.
- ✓ Fine-tuning is fundamentally different from prompting and prompt tuning.
- ✓ LoRA/QLoRA make adaptation substantially more practical under limited compute.
- ✓ SFT connects supervised input/output examples to the language-model training objective.
- ✓ Training metrics are useful, but final evaluation must measure the behavior we actually care about.
- ✓ The next session turns these concepts into a complete, hands-on Colab workflow.



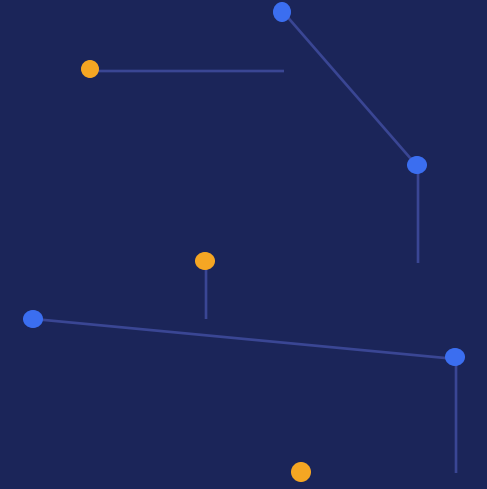
Up next: from theory to practice — building a real LoRA fine-tuning pipeline end to end.

FACULTY DEVELOPMENT PROGRAM

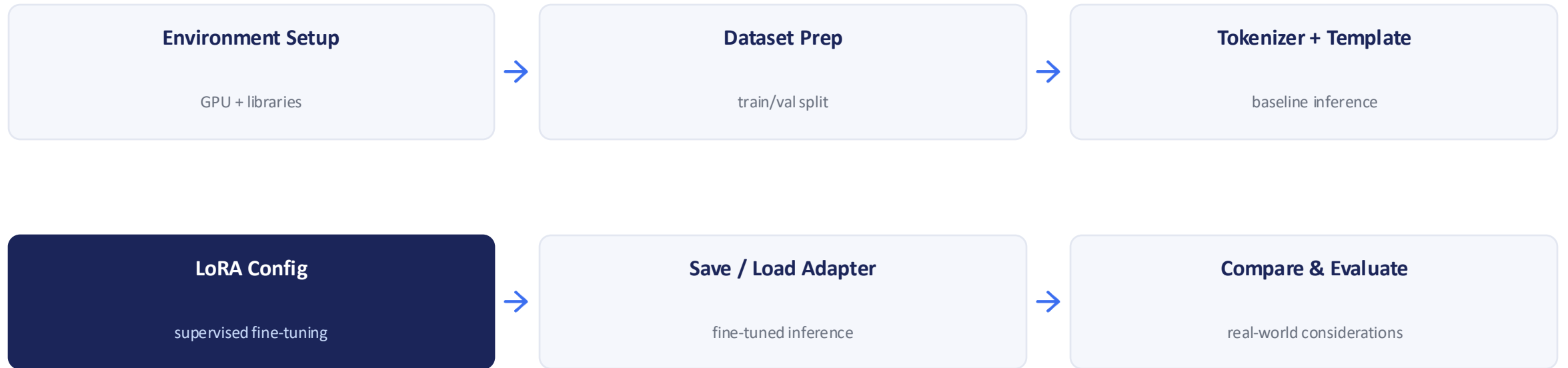
Hands-on LLM Fine-Tuning with LoRA

From dataset preparation to model evaluation using Google Colab

Session 2 | 90-minute hands-on workshop



Hands-on Roadmap



Learning Outcomes

1

Set up a GPU-based LLM fine-tuning environment in Google Colab.

Verify hardware and install the core stack.

2

Prepare and validate a supervised fine-tuning dataset.

Inspect structure, size and quality.

3

Understand tokenization and chat templates.

Convert conversations into model-ready inputs.

4

Generate a baseline response from a pretrained LLM.

Establish a fair before/after comparison.

5

Configure LoRA and perform supervised fine-tuning.

Attach adapters and run the training loop.

6

Save/reload a LoRA adapter and compare base vs fine-tuned outputs.

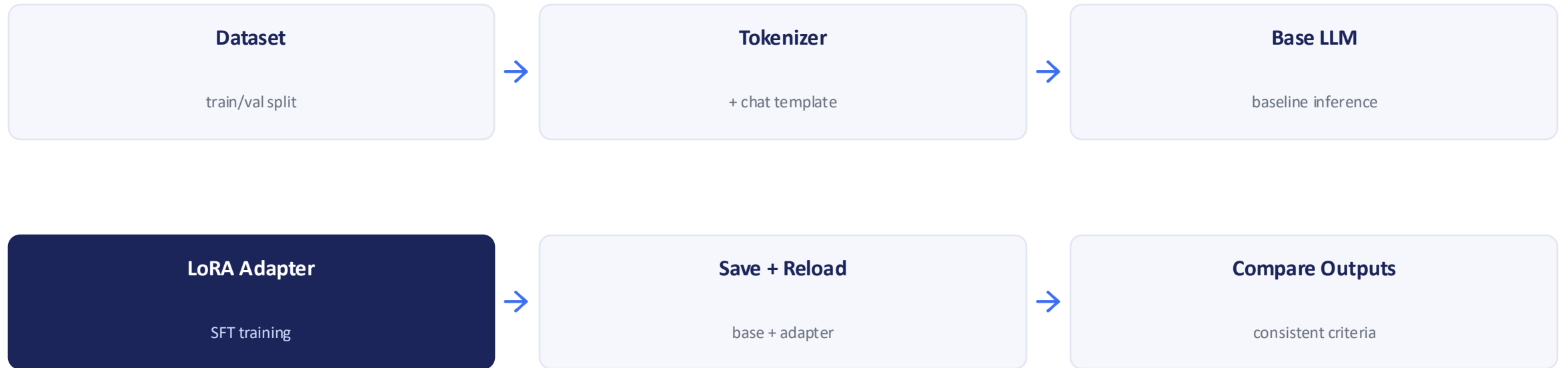
Reproduce results reliably.

7

Understand how dataset quality and evaluation affect conclusions.

Avoid over-trusting a single metric.

Hands-on Architecture: What Are We Building?



Google Colab & GPU Environment Setup

- Platform: Google Colab with an available NVIDIA GPU runtime.
- Core stack: PyTorch, Hugging Face Transformers, Datasets, PEFT, TRL and Accelerate.
- Verify CUDA availability and GPU name before loading the model.
- Install compatible library versions and keep the environment reproducible.
- Then load the tokenizer and the base instruction-tuned LLM.



setup.py

```
import torch
print("PyTorch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))

!pip install -q -U transformers datasets \
    peft trl accelerate
```

Pretrained Model vs Instruction-Tuned Model

From learning general language patterns → learning to follow human instructions

PRETRAINED LLM

- Learns from large-scale text
- Main objective: next-token prediction
- Learns language patterns, facts and representations
- General-purpose text completion
- Not specifically optimized for instruction following



INSTRUCTION-TUNED LLM

- Starts from a pretrained model
- Further trained on instruction → response examples
- Learns how to respond to user requests
- Better aligned with helpful task completion
- Designed for conversational / instructional use

Training progression

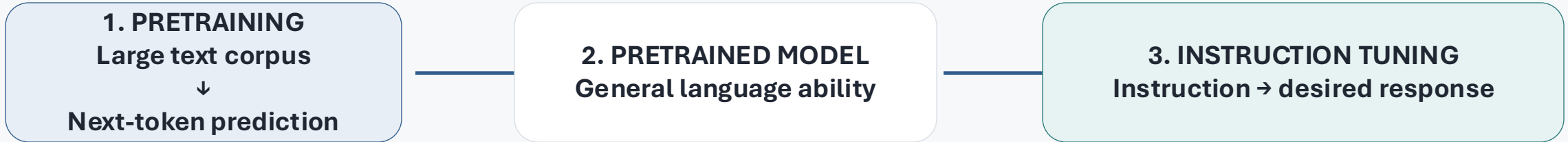
Large text corpus

Pretrained LLM

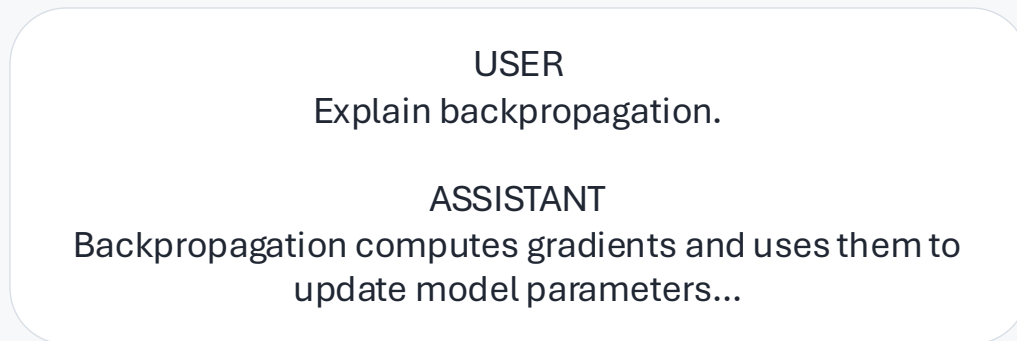
Instruction → Response

What Does "Instruct" Actually Mean?

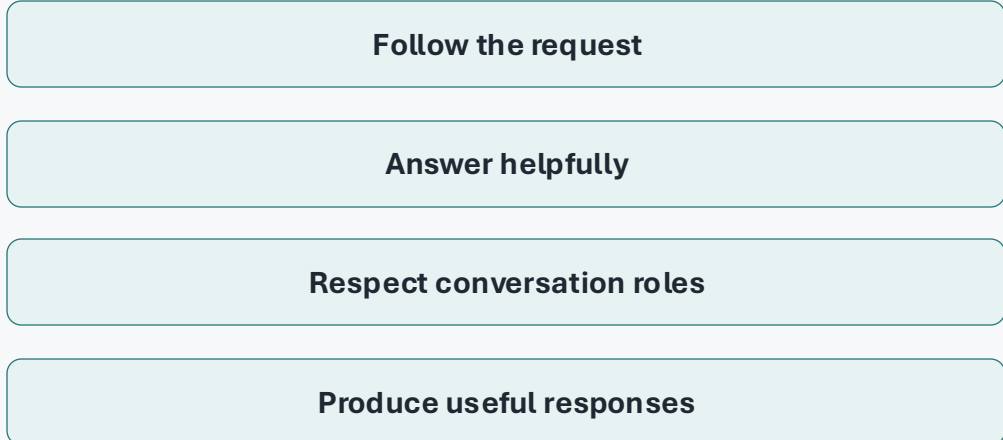
Instruction tuning is an additional training stage — not training an LLM from scratch.



Example instruction-tuning data



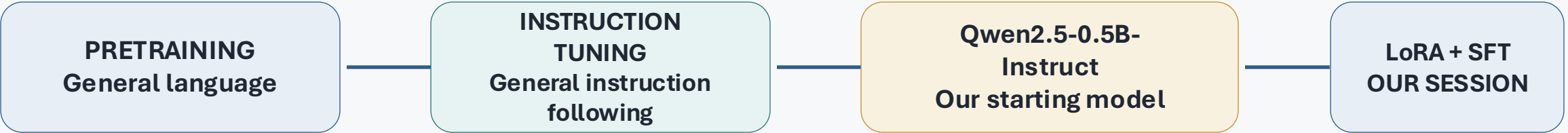
What behavior does this teach?



Key idea: an Instruct model is still the same kind of language model — it has undergone an additional training stage that shapes its behavior.

Our Starting Point — We Are NOT Teaching Instruction Following From Scratch

We start with an instruction-tuned model and specialize its behavior for our Deep Learning assistant.



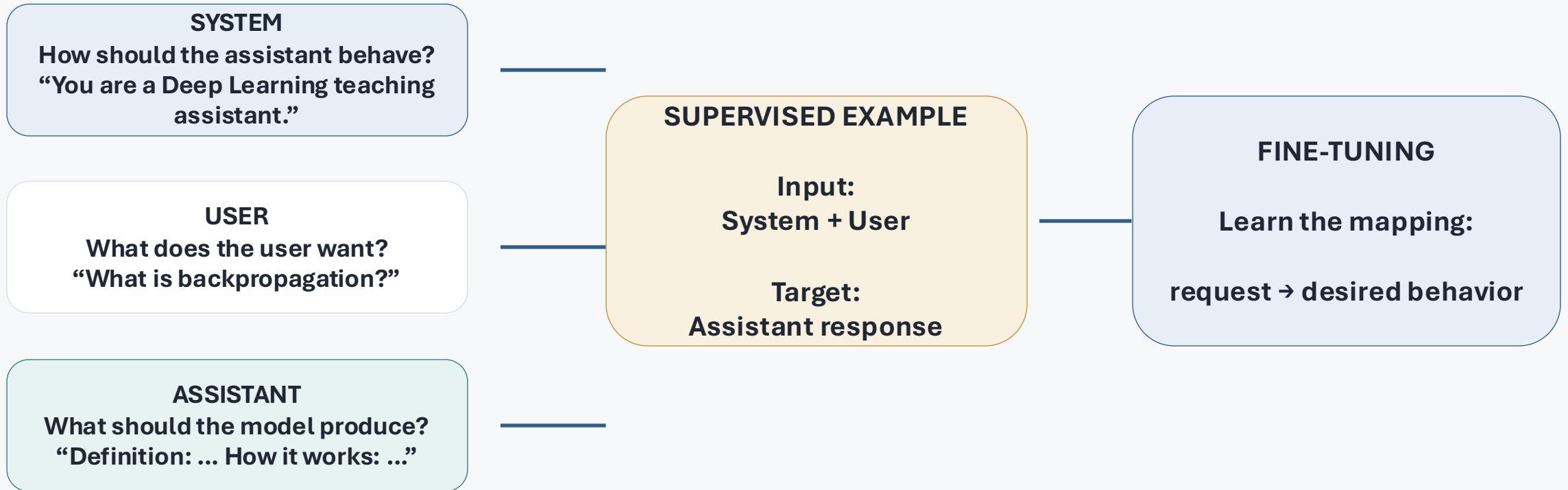
What does our fine-tuning add?

Already has	General language ability + basic instruction following
We teach	Deep Learning teaching style
We want	Definition → How it works → Why it matters → Example
Method	Supervised examples + LoRA adapter

Instruction tuning → general instruction following
Our fine-tuning → specialized behavior

How This Leads to System / User / Assistant Data

Our dataset gives the instruction-tuned model examples of the behavior we want.



Next: Chat Template → converts this structured conversation into the model-specific format expected by the Instruct model.

Dataset: From Raw JSON to Training Examples

- Deep Learning Q&A dataset for supervised fine-tuning.
- Represent conversations using system, user and assistant roles.
- Training examples teach both content and desired response behavior.
- Quality, consistency, diversity and correct formatting are critical.
- Before tokenization, inspect examples and verify the schema.

```
sample.json

{
  "messages": [
    {"role": "system",
     "content": "You are a Deep Learning
                teaching assistant."},
    {"role": "user",
     "content": "What is backpropagation?"},
    {"role": "assistant",
     "content": "Backpropagation computes
                gradients..."}
  ]
}
```

Dataset Inspection & Train/Validation Split

- Load the JSON and inspect the first examples.
- Verify the message structure and overall dataset size.
- Create a reproducible train/validation split using a fixed seed.
- Training data updates parameters; validation data monitors generalization.
- Checkpoint: do not proceed until examples in both splits look correct.

```
split.py

from datasets import load_dataset

dataset = load_dataset("json",
                        data_files="deep_learning_sft_dataset"
                                "_improved.json")
dataset = dataset["train"].train_test_split(
    test_size=0.15, seed=42)

train_dataset = dataset["train"]
val_dataset = dataset["test"]

print("Train:", len(train_dataset))
print("Validation:", len(val_dataset))
```

Tokenization: From Text to Token IDs

- LLMs consume numerical token IDs rather than raw human-readable text.
- Tokens can be words, sub-words or punctuation.
- Tokenization affects sequence length, memory use and context-window consumption.
- The tokenizer must match the model's vocabulary and expected special tokens.
- Chat templates then provide the model-specific conversational formatting.

```
tokenize.py

from transformers import AutoTokenizer

MODEL_NAME = "Qwen/Qwen2.5-0.5B-Instruct"
tokenizer = AutoTokenizer.from_pretrained(
    MODEL_NAME)

text = "What is backpropagation?"
print(tokenizer.tokenize(text))
print(tokenizer.encode(text))
```

Chat Templates: Formatting Conversations

- Structured system/user/assistant messages must be converted to the format the instruction-tuned model expects.
- Use the tokenizer's chat template rather than manually inventing model-specific control tokens.
- Training includes the target assistant response; inference typically ends with a generation prompt.
- Consistent formatting across training and inference is essential for correct behavior.

```
chat_template.py

formatted = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=False
)
print(formatted)

prompt = tokenizer.apply_chat_template(
    prompt_messages,
    tokenize=False,
    add_generation_prompt=True
)
```

Baseline Inference: Evaluate Before Fine-Tuning

- Before changing model parameters, establish what the base model already does.
- Use representative evaluation prompts that will remain fixed for the final comparison.
- Evaluate correctness, relevance, clarity, conciseness, terminology and teaching style.
- Save the baseline responses so the post-training comparison is reproducible.

```
baseline.py

from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME, torch_dtype="auto",
    device_map="auto"
)

baseline_prompts = [
    "What is backpropagation?",
    "Why do neural networks need "
    "activation functions?",
    "CNNs vs Transformers?",
    "Explain overfitting and dropout.",
    "Role of the learning rate?"
]
```

Why Parameter-Efficient Fine-Tuning?



Full fine-tuning is costly

Updates most/all model parameters — expensive in both memory and storage.



PEFT freezes the backbone

Freezes most pretrained parameters and trains only a small adaptation.



Prompting vs tuning

Prompt engineering changes instructions at inference time; prompt tuning learns continuous prompt parameters instead.



LoRA is our hands-on method

Learns low-rank updates to selected model layers — efficient and effective.



Practical on limited hardware

PEFT makes small-scale experimentation practical on constrained, single-GPU hardware like Colab.

Configuring LoRA

- LoRA rank r controls the capacity of the low-rank update.
- `lora_alpha` scales the adapter contribution; dropout provides regularization.
- `target_modules` identify the model layers where adapters are inserted.
- Exact target module names depend on the architecture and should be verified.
- After attaching LoRA, inspect trainable parameters to see how much of the model is actually updated.

```
lora_config.py

from peft import LoraConfig, get_peft_model

lora_config = LoraConfig(
    r=8, lora_alpha=16,
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=["q_proj", "k_proj",
                   "v_proj", "o_proj"]
)

model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
```

LoRA Configuration — What Do These Parameters Control?

Our configuration controls adapter capacity, scaling, regularization and where the adapter is inserted.

OUR CONFIGURATION

$r = 8$

$\text{lora_alpha} = 16$

$\text{lora_dropout} = 0.05$

$q_proj, k_proj,$
 v_proj, o_proj

r — RANK

Controls adapter capacity.
Higher $r \rightarrow$ more trainable parameters & capacity.

lora_alpha — SCALING

Controls the strength of the LoRA update.
Common scaling: α / r .

lora_dropout — REGULARIZATION

Randomly drops adapter activations during training.
Helps reduce overfitting.

target_modules — WHERE?

Layers where LoRA is inserted.
Here: Q, K, V and O projections in attention.

RANK: CAPACITY

$r = 4$

$r = 8$

$r = 16$

lower capacity $\leftarrow$ $\rightarrow$ higher capacity

OUR SCALING

$\alpha / r = 16 / 8 = 2$
LoRA contribution $\times 2$

ATTENTION TARGETS

Q

K

V

O

q_proj k_proj v_proj o_proj

Key idea: r = how much can be learned | α = how strongly it is applied | dropout = regularization | target_modules = where it is learned

Supervised Fine-Tuning (SFT)

- SFT trains on examples where the desired response is known.
- Forward pass → token predictions → loss → backpropagation → LoRA parameter update.
- Base model weights remain frozen; only LoRA parameters receive gradients.
- Training configuration includes epochs, batch size, gradient accumulation and learning rate.
- Validation data is used throughout to monitor generalization.

```
train.py

from trl import SFTConfig, SFTTrainer

training_args = SFTConfig(
    output_dir="./deep_learning_lora",
    num_train_epochs=2,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=4,
    learning_rate=2e-4,
    eval_strategy="epoch",
    save_strategy="epoch",
)

trainer = SFTTrainer(
    model=model, args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    processing_class=tokenizer
)

trainer.train()
```

Training Monitoring & Interpreting Results



Training loss

Indicates how well the model predicts training targets — should trend downward.



Epoch vs. global step

Epoch = one pass through the training data; global step = optimizer update count.



Validation loss

Measured on unseen examples without updating parameters — the real generalization signal.



Effective batch size

Approximately per-device batch size × gradient accumulation steps.



Reading the trend

Train ↓ + validation ↓ is generally encouraging; train ↓ + validation ↑ can indicate overfitting.



Loss is diagnostic, not final

Loss trends are useful signals, but output quality ultimately determines usefulness.

Saving & Loading the LoRA Adapter

- Save the trained adapter rather than creating a full new copy of the base LLM.
- `adapter_model.safetensors` stores learned weights; `adapter_config.json` stores configuration.
- Reload the base model and attach the adapter with `PeftModel.from_pretrained`.
- Adapters can be stored, transferred and reused with the same base model.
- During inference, parameters are used but never updated.

```
ADAPTER_PATH = "./deep_learning_lora_adapter"
trainer.save_model(ADAPTER_PATH)

from peft import PeftModel
base_model = AutoModelForCausalLM \
    .from_pretrained(
        MODEL_NAME, torch_dtype="auto",
        device_map="auto"
    )
finetuned_model = PeftModel.from_pretrained(
    base_model, ADAPTER_PATH
)
finetuned_model.eval()
```

Base Model vs Fine-Tuned Model

- Use exactly the same evaluation prompts before and after fine-tuning.
- Generate and save baseline responses first, then generate fine-tuned responses.
- Compare correctness, relevance, clarity, conciseness, terminology, style and consistency.
- A different answer is not automatically a better answer.
- The goal is desired behavior improvement, not simply output change.



compare.py

```
for prompt in baseline_prompts:
    print("=" * 60)
    print("QUESTION:", prompt)
    print("BASE MODEL:",
          baseline_results[prompt])
    print("FINE-TUNED MODEL:",
          finetuned_results[prompt])
```

Evaluating Fine-Tuned LLMs



Training metrics

Loss, validation loss and perplexity — quick automatic signals from the training run itself.



Task-specific metrics

Accuracy, F1, exact match or other appropriate task measures.



Qualitative evaluation

Correctness, relevance, clarity, consistency and style, assessed by reading outputs.



Human evaluation

Important for open-ended generation where automatic metrics fall short.



LLM-as-a-judge

Can scale evaluation but can introduce bias — should be validated against human judgment.



For this workshop

Use a simple base-vs-fine-tuned rubric on a fixed set of prompts.

From Colab Experiment to Real-World Fine-Tuning



Real datasets are bigger

Real projects use larger, more diverse and carefully curated datasets than a workshop example.



Engineering decisions scale up

Model choice, compute, sequence length, LoRA/QLoRA settings and evaluation all become real engineering decisions.



Production needs more

Production systems need safety, privacy, robustness, monitoring and deployment considerations.



Consider RAG

Consider RAG when the primary need is access to changing external knowledge.



Fine-tuning's sweet spot

Most useful for specialized behavior, task execution, style or domain response patterns.



They can combine

RAG and fine-tuning can also be combined for the best of both approaches.

Session 2 Summary & Take-Home Checklist

- ✓ Prepare an SFT dataset and train/validation split.
- ✓ Understand tokenization and chat templates.
- ✓ Establish a baseline before training.
- ✓ Configure LoRA and perform SFT.
- ✓ Save/reload adapters and reproduce inference.
- ✓ Compare base vs fine-tuned outputs using fixed prompts.
- ✓ Interpret training/evaluation behavior and identify when fine-tuning vs RAG is appropriate.



Key takeaway: Don't just fine-tune a model. Fine-tune it for a reason — and measure whether it worked.